Private Network Service Platform

Team 1 — Solo Phase 1 Architecture Document

Declaration

Private Network Service Platform - Team 1 (solo). All four roles (DNS, edge/load balancer, Backend A, Backend B) run on one MacBook using loopback aliases 127.0.0.11–127.0.0.14. Protocol traffic is real and captured with Wireshark on lo0.

1. Architecture Overview

The Phase 1 system models a four-machine network on a single Mac by assigning separate loopback IP addresses to each role. The client uses the private domain **app.team1.test**. DNS resolves that name to the edge address, nginx terminates HTTPS and load-balances requests between two HTTP backends, and the backends expose status and information endpoints.

2. Machine Roles and Service Inventory

Role	Cloud equivalent	IP	Service	Port / Protocol
Mac 1 — DNS	AWS Route 53	127.0.0.11	dnsmasq	53 / UDP + TCP
Mac 2 — Edge + LB	AWS ALB / CDN edge	127.0.0.12	nginx	80, 443 / TCP
Mac 3 — Backend A	EC2 instance A	127.0.0.13	backend.py A	3001 / TCP
Mac 4 — Backend B	EC2 instance B	127.0.0.14	backend.py B	3002 / TCP

3. Actual Host Network Details

All four logical roles are hosted on the same physical Mac. The host's Wi-Fi interface and network values recorded during Phase 1 are:

Property	Recorded value
Interface	en0 (Wi-Fi)
IPv4	10.7.23.77
Subnet mask	255.255.224.0
Gateway	10.7.0.1
Loopback role addresses	127.0.0.11–127.0.0.14

4. Request Flow

1. The client requests **app.team1.test** and the macOS resolver directs the **team1.test** domain to the local dnsmasq server at 127.0.0.11.
2. dnsmasq returns 127.0.0.12 for app.team1.test (and api.team1.test).
3. The client opens a TCP connection to the nginx edge on port 443.
4. nginx performs TLS termination using the locally trusted Team 1 certificate.
5. nginx forwards the HTTP request over the loopback interface to the upstream backend pool.
6. The upstream pool contains Backend A at 127.0.0.13:3001 and Backend B at 127.0.0.14:3002.
7. nginx load-balances requests between the two backends and exposes the selected backend through the X-Backend response header.
8. If an upstream fails, nginx can try the other backend; if both are unavailable, the edge returns HTTP 502 Bad Gateway.

5. Protocol and Layer Mapping

Protocol / Component	TCP/IP layer	OSI layer	Evidence
DNS	Application	7	dig; Wireshark dns
HTTP/1.1, HTTP/2	Application	7	curl -v; DevTools
TLS	Application / Transport boundary (brief)	5 & 6	Wireshark tls.handshake
TCP	Transport	4	SYN / SYN-ACK / ACK; port 443
UDP	Transport	4	DNS port 53
IPv4	Internet	3	127.0.0.x; ping
Loopback / Wi-Fi	Link	2 (+ Physical)	lo0; MAC/interface

6. Private DNS Design

The project uses the private **.test** namespace. macOS is configured with a resolver file for team1.test pointing to 127.0.0.11. dnsmasq listens on 127.0.0.11 and provides local records for app.team1.test and api.team1.test. The edge address returned is 127.0.0.12, so clients do not directly discover the backend addresses through DNS.

7. Edge, TLS and Load Balancing

nginx listens on 127.0.0.12:80 and redirects HTTP requests to HTTPS. On port 443 it terminates TLS for app.team1.test and api.team1.test. The certificate uses SAN entries for both names and is signed by the Team 1 local root CA. The nginx upstream contains both backend servers and uses round-robin selection by default. Failure handling is configured with max_fails, fail_timeout, and proxy_next_upstream.

8. HTTP Caching

The /api/info endpoint is intentionally cacheable. It returns Cache-Control: public, max-age=60 and an ETag. A client that sends If-None-Match with the current ETag receives HTTP 304 Not Modified, while an old or mismatching ETag results in a normal HTTP 200 response. Browser DevTools evidence and curl output document the cache behavior.

9. Packet Analysis

Wireshark captures are taken on lo0. Phase 1 evidence includes DNS queries, the TCP handshake on port 443, TLS 1.2 and TLS 1.3 handshakes, encrypted application data, port/conversation views, a flow graph, and the readable plaintext HTTP hop between nginx and the backend. This demonstrates that TLS is terminated at the edge and that the edge-to-backend hop is HTTP.

10. Failure Scenarios

Five controlled failures were demonstrated: an incorrect DNS server, an incorrect DNS record, one backend being unavailable, both backends being unavailable, and a request to an incorrect port. These tests separate DNS, TCP/TLS reachability, reverse-proxy behavior, and backend availability as distinct troubleshooting layers.

11. Cloud Mapping

The local architecture maps conceptually to a cloud deployment: the private DNS role corresponds to AWS Route 53, the nginx edge corresponds to an AWS Application Load Balancer or CDN edge node, and the two backend roles correspond to separate EC2 application servers. The loopback deployment is the Phase 1 implementation of this logical architecture on one Mac.